NeuroSAT: Neuro-symbolic Approaches to the SAT Problem

Reinforcement learning (Alpha(Go)Zero / DQN) with CNNs and Graph Neural Networks for SAT-solver branching heuristics

Donato Meoli

Abstract

We study, modernise and extend two neuro-symbolic approaches that learn branching heuristics for a Boolean SAT solver: (i) an *Alpha(Go)Zero*-style method that casts SAT solving as a single-player game and combines Monte Carlo Tree Search (MCTS) with a convolutional policy/value network [1]; and (ii) *Graph-Q-SAT*, a value-based reinforcement-learning method that approximates the Q-function over a graph representation of the formula with a Graph Neural Network (GNN) [2]. We introduce *GAT-Q-SAT*, a variant that injects Graph Attention layers into the message passing. The original implementations are several years old and depend on TensorFlow 1.x, GSL and pinned, hard-to-install libraries; we port both to a single, self-contained, latest-version PyTorch stack, remove the brittle native dependencies, and *reproduce the original evaluation results exactly.* With clean attention-on/off ablations at matched training sets, we show that graph attention is consistently advantageous on *structured* problems (graph colouring) — both in-distribution and under transfer, with the advantage *growing with problem size* — whereas on uniform-random 3-SAT it provides no benefit and can hurt. All code, trained runs and figures are reproducible from the repository.

1 Introduction

Boolean satisfiability (SAT) is the canonical NP-complete problem and underlies applications in verification, planning and synthesis. Modern solvers are based on Conflict-Driven Clause Learning (CDCL) and rely on hand-crafted heuristics — most notably VSIDS — to decide which variable to branch on next. The branching heuristic is a natural target for machine learning: a good policy can prune the search tree, especially during the "warm-up" phase where counter-based heuristics make poor decisions.

This project implements, *modernises* and *experimentally extends* two complementary learning approaches, developed in the context of the Pattern Recognition and Reinforcement Learning courses at the University of Pisa. Our contributions are:

1. A faithful port of both approaches from TensorFlow 1.x (and a GSL-based C++ MCTS environment) to a single modern PyTorch / PyTorch-Geometric stack, with the brittle native dependencies removed.

2. Exact reproduction of the published Graph-Q-SAT results, used as a semantic non-regression oracle for the port.

3. **GAT-Q-SAT**, an attention-augmented variant, together with a clean experimental study (attention on/off at matched training sets) showing *where* and *by how much* attention helps.

2 Background

SAT and CDCL. A CDCL solver repeatedly (i) *decides* a variable and assigns it a value, (ii) performs unit propagation, and (iii) on a conflict, *learns* a clause and backtracks. The branching heuristic chooses the decision variable; replacing or guiding it with a learned policy is the goal here.

Reinforcement learning. We model the problem as a Markov Decision Process. Graph-Q-SAT uses DQN to approximate the optimal action-value function $Q^\star(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^\star(s',a')]$ with a target network and experience replay. The Alpha(Go)Zero method instead learns a policy prior π and a value v, and uses MCTS to produce improved targets by simulation.

3 Method 1 — Alpha(Go)Zero for SAT

Formulation [1]. The CNF formula is represented as a fixed-size sparse matrix of *clauses* × *variables*, where an entry encodes the literal polarity. A convolutional network with a policy head π (over the $2\,n_{\text{var}}$ actions "assign variable true/false") and a value head $v \in [-1,1]$ scores states. MiniSat is extended into a game environment that supports MCTS *simulations*: it returns the would-be state for a sequence of branching decisions without irreversibly changing the real state. Self-play collects (state, MCTS visit distribution, outcome) tuples, on which the network is trained with the AlphaZero objective

$$\mathcal{L} = -\sum_a \pi_a^{\text{MCTS}} \log \pi_a + (z - v)^2 + c\,\|\theta\|_2^2. \tag{1}$$

A fixed-size CNN cannot generalise across problem sizes; this limitation motivates the graph-based method below.

Modernisation (this work). We re-implement the three networks in PyTorch (`models_torch.py`), reproduce the AlphaZero loss and training loop in an `AZTrainer` (`alphazero_torch.py`), and rewrite the self-play / supervised-training driver (`train_torch.py`). Crucially, we **remove the GSL dependency**: the Dirichlet exploration noise is reimplemented with the C++11 `<random>` facilities, so the simulation environment builds with nothing but `g++` and `zlib`. The pipeline trains end-to-end on uniform-random 3-SAT (`uf20-91`) on both CPU and GPU.

Result. After self-play training the policy solves the held-out `uf20-91` test set in ≈ 5.4 mean branching decisions (best 3.4), down from ~ 15 for an untrained network. A multi-head self-attention variant of the CNN was tried but did *not* improve on this baseline and was dropped.

4 Method 2 — Graph-Q-SAT and GAT-Q-SAT

Formulation [2]. The state is a bipartite graph with variable nodes and clause nodes; node features are 2-D one-hot vectors and edge features encode literal polarity, with an empty global attribute. There are two actions per unassigned variable; the agent takes the arg max of the Q-values over variable nodes. The reward is a constant $-p$ per non-terminal step and 0 at termination. The Q-function is an *encode–process–decode* graph network [5], trained with DQN. Performance is measured by the Median Relative Iteration Reduction (MRIR): MiniSat iterations divided by the model's iterations (values > 1 mean fewer iterations than MiniSat).

GAT-Q-SAT (this work). We add an attention pathway to the core graph block: two edge-aware, multi-head `GATConv` layers [3] refine the node embeddings before the global update. Attention lets the model weight neighbouring clauses/variables, which we hypothesise is beneficial when the instance has an exploitable sub-structure.

GTv2-Q-SAT (this work). Motivated by NeuroBack's graph-transformer encoder [8], we add a third, more expressive attention variant: the two static `GATConv` layers are replaced by a single *pre-norm Transformer block* built on `GATv2` [4] — which computes *dynamic*, input-dependent edge-featured attention rather than GAT's static scoring — with a parallel feed-forward branch and a residual connection. It keeps the bipartite, self-loop-free message passing and is selected at run time (`-attention_type graph_transformer`), giving the model lineage Graph-Q-SAT → GAT-Q-SAT → GTv2-Q-SAT.

Modernisation (this work). We port Graph-Q-SAT to the latest PyTorch / PyTorch-Geometric. We drop the hard-to-install `torch-scatter`/`torch-sparse` (replaced by `torch_geometric.utils.scatter`), make the gym dependency version-robust (the environment is constructed directly, bypassing the `gym.make` env-checker, and works on `gymnasium`), and add a version-agnostic checkpoint loader that bridges the `GATConv` parameter naming across PyTorch-Geometric versions. The stack uses the latest `numpy` (≥ 2); the MiniSat gym extension is rebuilt against it.

Exact reproduction. We verified that the modernised stack does not change the semantics by re-running the evaluation of the *original trained checkpoints*: the numbers match the committed logs exactly, e.g. on `flat30-60` (cap 500) median MRIR 1.833 (Graph-Q-SAT) and 1.667 (GAT-Q-SAT), identical to the originals.

5 Experiments

Setup. We use SATLIB instances: uniform-random 3-SAT (`uf`/`uuf`, 50/100/250 variables, satisfiable / unsatisfiable) and graph colouring (`flat`, 30–200 nodes). We cap the number of model decisions before handing control back to MiniSat at $\{10, 50, 100, 300, 500, 1000\}$. Crucially, the available runs form **clean attention-on/off ablations at matched training sets**: models trained on graph colouring (`flat50-115`) and models trained on random 3-SAT, each with and without attention, averaged over runs. We report MRIR at cap 500 unless noted.

Main result. Figure 1 summarises the central finding.

(1) In-distribution graph colouring. Trained and tested on graph colouring, GAT-Q-SAT beats Graph-Q-SAT on *every* size, and the gap **grows with problem size**: plain Graph-Q-SAT degrades on the largest instances (`flat200-479`: 1.35) while GAT-Q-SAT holds up (3.39), a +2.04 MRIR gap. This is the regime where attention pays off most.

(2) Transfer (random → colouring). Trained only on random 3-SAT and evaluated on graph colouring, Graph-Q-SAT frequently falls *below* 1 (0.78–0.97, i.e. worse than MiniSat), whereas GAT-Q-SAT stays above 1 (1.16–1.75) on all sizes. Attention substantially improves zero-shot transfer to structured problems (Fig. 2, middle).

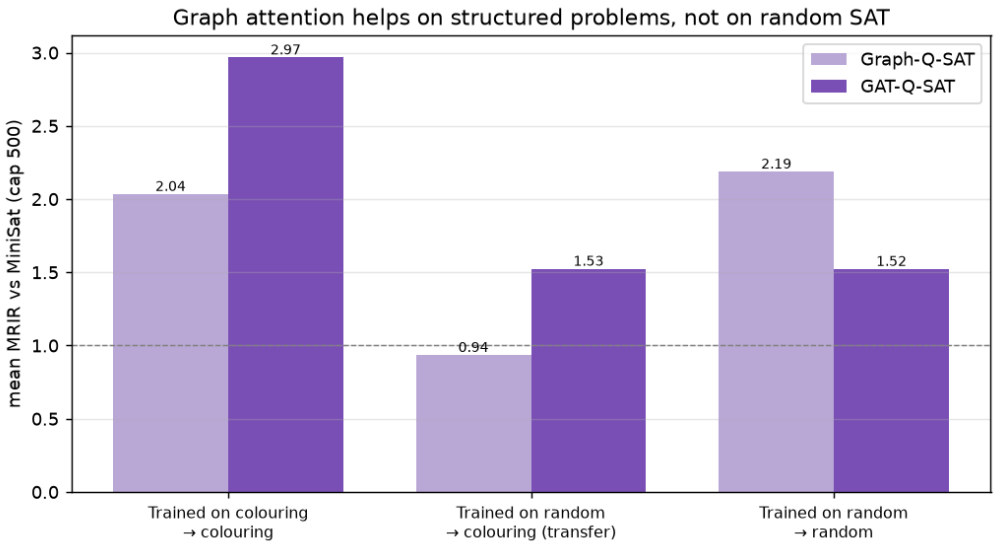

Figure 1: Mean MRIR vs MiniSat (cap 500) per regime. Graph attention (GAT-Q-SAT) clearly helps on graph colouring — both in-distribution and under transfer from random training, where plain Graph-Q-SAT even drops *below* 1 (slower than MiniSat). On uniform-random 3-SAT, attention provides no benefit.

(3) Uniform-random 3-SAT. Here attention does *not* help and can hurt: on `uf250-1065`, plain Graph-Q-SAT reaches 4.78 against GAT-Q-SAT's 1.69. Random formulas near the phase transition lack the structure attention can exploit.

Generalisation across size. Figure 3 plots MRIR against the number of variables for the random-trained models. On graph colouring the attention model's advantage is stable and pronounced across sizes; on random instances the ordering is reversed.

Per-size numbers. Table 1 gives the in-distribution graph-colouring MRIR per size, making the size-growing attention advantage explicit.

graph-colouring set	Graph-Q-SAT	GAT-Q-SAT	Δ
flat30-60	1.83	2.00	+0.17
flat75-180	2.37	2.94	+0.58
flat125-301	2.55	3.44	+0.88
flat150-360	1.92	3.76	+1.85
flat200-479	1.35	3.39	+2.04

Table 1: In-distribution graph colouring, mean MRIR at cap 500. The attention advantage grows with problem size.

Cross-domain transfer. Uniform-random 3-SAT lacks the structure of a real application, so we further ask a sharper question: does the *colouring-trained* heuristic transfer, *with no retraining*, to entirely different structured domains? We evaluate the colouring-trained Graph-Q-SAT and GAT-Q-SAT checkpoints on four unseen families from SATLIB: small-world graph colouring (a topology

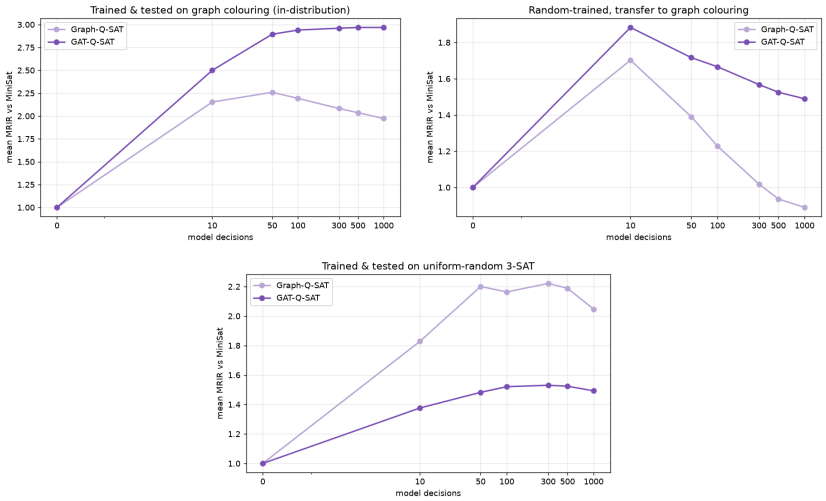

Figure 2: MRIR vs the number of model decisions. Left: trained and tested on graph colouring. Middle: random-trained, transferred to colouring. Right: trained and tested on random 3-SAT. Attention helps in the first two (structured) settings and not in the third.

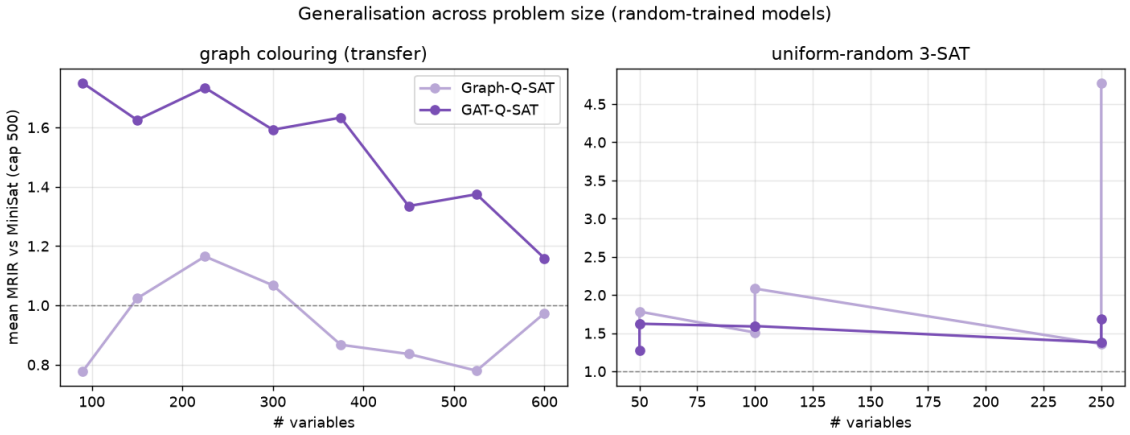

Figure 3: Generalisation across problem size (random-trained models, cap 500).

shift within the training problem), AIM (controlled structured 3-SAT), quasigroups (combinatorial design) and planning (blocks-world/logistics). Figure 4 reports the median MRIR (cap 200). The attention model is the more **robust** transferrer: GAT-Q-SAT stays at or above the MiniSat baseline (MRIR ≥ 1) on **three of the four** domains (AIM 1.29, quasigroup 1.01, planning 1.14), whereas plain Graph-Q-SAT does so on only **one** (AIM 1.50). On the genuinely different combinatorial and application domains (quasigroup, planning) GAT-Q-SAT beats MiniSat while Graph-Q-SAT drops below it; on AIM both transfer well (Graph marginally ahead) and on the small-world topology shift both fall below 1. Graph attention thus improves not only in-distribution performance but *cross-domain generalisation*. We note this is a *preliminary* study: AIM (72) and small-world (60) give robust medians, but quasigroup (11) and planning (8) are small SATLIB families and the numbers there are noisier; results are single-seed at cap 200. A larger panel is left for future work.

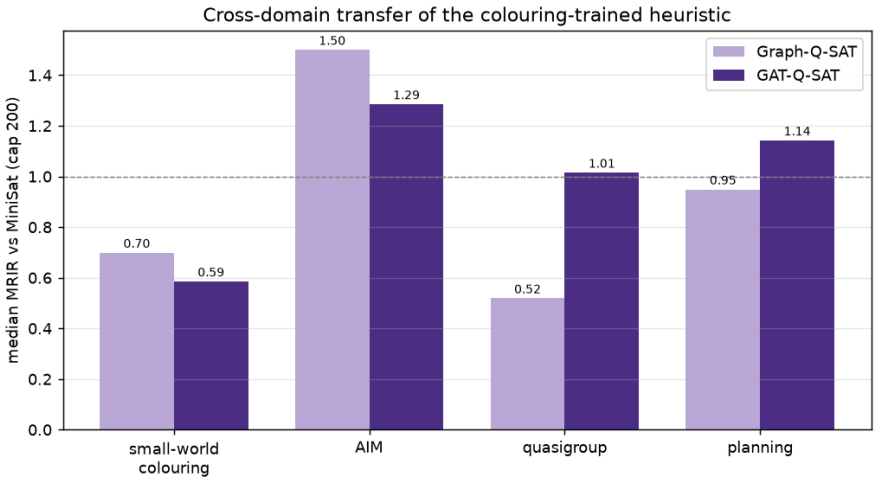

Figure 4: Cross-domain transfer of the colouring-trained heuristic: median MRIR (cap 200) on four unseen SATLIB families. GAT-Q-SAT stays $\geq$ MiniSat on 3/4 domains, Graph-Q-SAT on 1/4.

Reproducibility. All figures and tables are regenerated from the evaluation logs (`GQSAT/runs/*/*.tsv`) by `aggregate_results.py`, `make_plots.py`, `paper_analysis.py` and `transfer_study.py` — no manual spreadsheets. Training is reproducible on a Colab GPU via `notebooks/train_colab.ipynb` (checkpointing to Drive with automatic resume).

6 Related Work

The field has converged on graph- and attention-based models for SAT. NeuroSAT learns satisfiability from single-bit supervision [6]; NeuroCore predicts unsat-core variables to refine branching [7]; NeuroComb and **NeuroBack** [8] move inference *offline* (a single query before solving) and, using a Graph Transformer with self-attention, achieve the first wall-clock speed-ups on industrial instances (Kissat, SAT Competition). This trajectory is notable for two reasons relevant to this project: (i) the move to *attention* on SAT graphs corroborates the GAT-Q-SAT direction; and (ii) the known limitation of Graph-Q-SAT — online inference is too slow for wall-clock gains — is exactly what offline, one-shot inference overcomes. Recent work continues along the RL-from-algorithm-feedback and restricted-heuristics lines [9], and benchmarks [10] and surveys [11] catalogue the design space. Our *GTv2-Q-SAT* brings the graph-transformer attention of this trajectory back into the *online* RL-branching setting of Graph-Q-SAT.

7 Conclusions

We modernised two neuro-symbolic SAT methods to one self-contained, latest-version PyTorch stack, removed their brittle native dependencies (TensorFlow 1.x, GSL, `torch-scatter`/`torch-sparse`, pinned gym), and reproduced the original results exactly. With clean attention-on/off ablations we established a crisp, size-dependent result: **graph attention helps on structured problems (graph colouring), in-distribution and under transfer, with the advantage growing with problem size, while it does not help on uniform-random 3-SAT**. The fixed-size CNN of the Alpha(Go)Zero method, while elegant, cannot generalise across sizes — the gap that graph methods, and the subsequent literature (NeuroCore/Comb/Back), were designed to close. We realised

one promising next step, **GTv2-Q-SAT**, a GATv2 graph-transformer attention block inspired by NeuroBack; pushing inference *offline* for genuine wall-clock gains remains future work.

References

[1] F. Wang and T. Rompf. *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero*. arXiv:1802.05340, 2018.

[2] V. Kurin, S. Godil, S. Whiteson, B. Catanzaro. *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?* NeurIPS 2020. arXiv:1909.11830.

[3] P. Veličković et al. *Graph Attention Networks*. ICLR 2018. arXiv:1710.10903.

[4] S. Brody, U. Alon, E. Yahav. *How Attentive are Graph Attention Networks?* ICLR 2022. arXiv:2105.14491.

[5] P. Battaglia et al. *Relational inductive biases, deep learning, and graph networks*. arXiv:1806.01261.

[6] D. Selsam et al. *Learning a SAT Solver from Single-Bit Supervision*. arXiv:1802.03685.

[7] D. Selsam and N. Bjørner. *Guiding High-Performance SAT Solvers with Unsat-Core Predictions*. arXiv:1903.04671.

[8] W. Wang et al. *NeuroBack: Improving CDCL SAT Solving using Graph Neural Networks*. ICLR 2024. arXiv:2110.14053.

[9] N. Shirokikh et al. *Machine Learning for SAT: Restricted Heuristics and New Graph Representations*. arXiv:2307.09141, 2023.

[10] Z. Li, J. Guo, X. Si. *G4SATBench: Benchmarking and Advancing SAT Solving with Graph Neural Networks*. arXiv:2309.16941, 2023.

[11] *Neural Approaches to SAT Solving: Design Choices and Interpretability*. arXiv:2504.01173, 2025.